Graph Algorithms Lab: BFS, DFS, Dijkstra

LAB INSTRUCTION | any programming language | black and white print version

Tashkent Webster University | Data Structures 2 | Fall 2026

Student name	Muhammad Zayniddinov	**Student ID**	_____
Group	_____	Date	26 October 2026

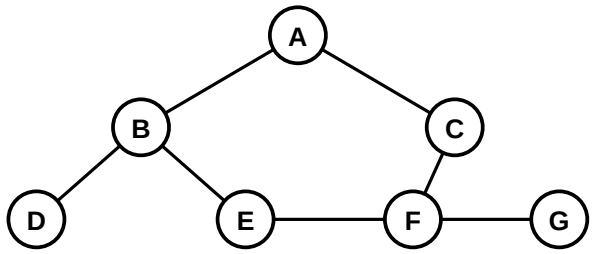
Goal

Implement three classic graph algorithms in **any programming language** you know (C, C++, Java, Python, C#, JavaScript, ...). You will do **3 tasks**: one for BFS, one for DFS and one for Dijkstra. Each task has a hand trace on paper, a program, and short written answers.

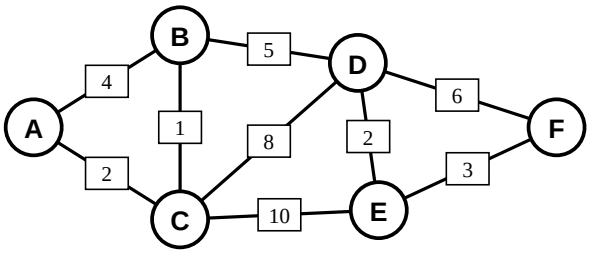
Rules

- Build the queue and the stack yourself (an array, list or linked list). Do not use a ready-made graph library.
- The graph data is given below. You may type it directly into your program.
- Examine neighbours in **alphabetical order** (A before B). Your output must match the expected output **exactly**, line by line.
- Write your own code. Add comments that explain the queue/stack and the main steps.

The two example graphs



Graph 1 (Tasks 1 and 2), undirected, no weights.
Edges: A-B, A-C, B-D, B-E, C-F, E-F, F-G.



Graph 2 (Task 3), undirected, weights are minutes/cost.
Edges: A-B 4, A-C 2, B-C 1, B-D 5, C-D 8, C-E 10, D-E 2, D-F 6, E-F 3.

Neighbour lists (Graph 1)

Node	**A**	**B**	**C**	**D**	**E**	**F**	**G**
Neighbours	B, C	A, D, E	A, F	B	B, F	C, E, G	F

Points

Task	**Algorithm**	**Points**
1	BFS - shortest path (fewest edges)	30
2	DFS - iterative, with an explicit stack	30
3	Dijkstra - cheapest paths	40
	Total	100

Task 1 - Breadth-First Search (30 points)

Idea. BFS visits nodes level by level using a **queue** (first in, first out). Mark a node as discovered when you **put it in the queue**. Because BFS counts edges, the first time it reaches a node is along a path with the **fewest edges**; the *parent* array lets you rebuild that path.

Pseudocode

```
BFS(graph, start):
    create an empty QUEUE Q
    mark start as discovered;  dist[start] = 0;  parent[start] = none
    enqueue(Q, start)
    while Q is not empty:
        u = dequeue(Q)                      // take from the FRONT
        visit u                             // print it
        for each neighbour v of u (A to Z):
            if v is not discovered:
                mark v as discovered
                dist[v] = dist[u] + 1;  parent[v] = u
                enqueue(Q, v)               // add at the BACK
```

A. Hand trace on Graph 1, start A (8 points)

Step	Node dequeued	Newly enqueued	Queue after the step (front ... back)
init	-	A	A
1	A	B, C	B, C
2	B	D, E	C, D, E
3	C	F	D, E, F
4	D	-	E, F
5	E	-	F
6	F	G	G
7	G	-	empty

Visit order: A B C D E F G

B. Program (18 points)

Write a program that runs BFS on Graph 1 and prints the lines below. The shortest-path lines must be computed with *parent* and *dist* (rebuild the path from the target back to the start). Handle the case 'no path'.

```
BFS order from A: A B C D E F G
Shortest A -> G: distance 3, path A -> C -> F -> G
Shortest D -> G: distance 4, path D -> B -> E -> F -> G
```

C. Short answers (4 points)

- Why do we mark a node when it is **enqueued** and not when it is dequeued?
- What is the running time of BFS (V nodes, E edges)? Why does BFS fail to give cheapest paths on a weighted graph?

1. Marking on enqueue prevents duplicate queue entries and keeps the first shortest-path parent.
2. BFS takes O(V + E) time with adjacency lists: each node is queued once and each edge is examined at most twice in an undirected graph. It minimizes edge count, not total weight.

Task 2 - Depth-First Search (30 points)

Idea. DFS goes as deep as possible along one branch, then **backtracks**. Use an explicit **stack** (last in, first out). A node is marked when it is **popped**, so the same node can be in the stack twice and the second copy is **skipped**. Pushing neighbours in **reverse** order makes the alphabetically first neighbour come out first.

Pseudocode

```
DFS(graph, start):
    create an empty STACK S
    push(S, start)
    while S is not empty:
        u = pop(S)                          // take from the TOP
        if u is already visited:  continue  // skip the second copy
        mark u as visited;  visit u         // print it
        for each neighbour v of u (Z down to A):     // REVERSE order
            if v is not visited:
                push(S, v)
```

A. Hand trace on Graph 1, start A (8 points)

Step	Node popped	Action	Pushed (in this order)	Stack after (top first)
init	-	Initialize	A	A
1	A	Visit	C, B	B, C
2	B	Visit	E, D	D, E, C
3	D	Visit	-	E, C
4	E	Visit	F	F, C
5	F	Visit	G, C	C, G, C
6	C	Visit	-	G, C
7	G	Visit	-	C
8	C	Skip visited	-	empty

Visit order: A B D E F C G

B. Program (18 points)

Write DFS with an **explicit stack** (no recursion) and print the lines below. Use the second start node G to prove that your program really works for any start.

```
DFS order from A: A B D E F C G
DFS order from G: G F C A B D E
```

C. Short answers (4 points)

- Why can a node be in the stack twice? In which step of your trace is a copy skipped?
- Compare the BFS order and the DFS order on Graph 1. Which data structure explains the difference?

1. A node is marked only when popped. Before that, another visited node may push another copy. At step 5, F pushes C even though C is already pending. The second C is skipped at step 8 because C was visited at step 6.
2. BFS: A B C D E F G. DFS: A B D E F C G. The FIFO queue explores levels; the LIFO stack follows a branch first. Reverse pushes make the smallest unvisited neighbour pop first.

Task 3 - Dijkstra's Algorithm (40 points)

Idea. Dijkstra finds the cheapest path from one source to every node when all weights are **non-negative**. Keep **dist** (best known cost, INF at the start, 0 for the source) and **prev** (node before it on that path). Repeatedly **settle** the unsettled node with the smallest dist, then **relax** its edges.

Pseudocode

```
Dijkstra(graph, source):
    dist[v] = INFINITY and prev[v] = none for every node v
    dist[source] = 0
    repeat while some unsettled node has dist < INFINITY:
        u = the unsettled node with the SMALLEST dist
        settle u                          // its dist is now final
        for each edge (u, v, w) where v is not settled:
            if dist[u] + w < dist[v]:       // relaxation
                dist[v] = dist[u] + w
                prev[v] = u
```

A. Hand trace on Graph 2, source A (10 points)

Step	Node settled (its dist)	Updates made (node = new dist via previous node)
init	-	A = 0; others = INF; all prev = none
1	A (0)	B = 4 via A; C = 2 via A
2	C (2)	B = 3 via C; D = 10 via C; E = 12 via C
3	B (3)	D = 8 via B
4	D (8)	E = 10 via D; F = 14 via D
5	E (10)	F = 13 via E
6	F (13)	No updates

Node	A	B	C	D	E	F
Final dist	0	3	2	8	10	13
Final prev	none	C	A	B	D	E

B. Program (24 points)

Implement Dijkstra on Graph 2 (a simple search for the minimum is enough; a heap is a bonus). Print one line after each settled node, then the cheapest path to every node.

```
init          | A=0 B=INF C=INF D=INF E=INF F=INF
settle A (0)  | A=0 B=4 C=2 D=INF E=INF F=INF
settle C (2)  | A=0 B=3 C=2 D=10 E=12 F=INF
settle B (3)  | A=0 B=3 C=2 D=8 E=12 F=INF
settle D (8)  | A=0 B=3 C=2 D=8 E=10 F=14
settle E (10) | A=0 B=3 C=2 D=8 E=10 F=13
settle F (13) | A=0 B=3 C=2 D=8 E=10 F=13
A to B: cost 3: A -> C -> B
A to C: cost 2: A -> C
A to D: cost 8: A -> C -> B -> D
A to E: cost 10: A -> C -> B -> D -> E
A to F: cost 13: A -> C -> B -> D -> E -> F
```

C. Short answers (6 points)

- Explain **relaxation** with one line of your table (for example, why B changes from 4 to 3).
- Why does Dijkstra give wrong results when an edge has a negative weight? How is BFS related to Dijkstra when every weight is 1?

Answers to Task 3 C are on page 5.

Submission checklist

- One source file per task (or one file with three clearly marked parts) in the language of your choice.
- The output of your programs matches the expected output exactly.
- Hand-trace tables and short answers are completed (on this sheet or on separate pages).
- Comments explain the queue / stack and the relaxation step. Your name and ID are at the top of every file.
- Upload to the course page before the deadline given in class.

Hints for any language

- A **queue** needs two indexes (front, back) or a built-in deque. A **stack** needs one index (top) or a list with push/pop at the same end.
- Map the letters A, B, C, ... to the numbers 0, 1, 2, ... and keep an array of names for printing.
- Use a very large number as INFINITY, and a special value (for example -1) for 'no previous node'.
- Execute your algorithm by hand on paper first. Print the queue / stack inside the loop while debugging.
- Test with a second start node. A program that only prints the expected text for one start does not earn the points.

Task 3 C - Short answers

1. Relaxation replaces a best-known cost when a cheaper route is found. After C is settled at cost 2, the route A -> C -> B costs 2 + 1 = 3, which is less than 4. Therefore dist[B] changes to 3 and prev[B] becomes C.

2. Dijkstra assumes that a settled distance cannot improve later. A negative edge can produce a cheaper route to an already settled node, so that assumption fails. In an undirected graph, a negative edge also gives a negative cycle if repeated walks are allowed. When all weights are 1, path cost equals the number of edges. BFS then finds the same shortest distances as Dijkstra using a FIFO queue instead of a minimum-distance selection.